

CLASSIC APACHE SPARK WORD COUNT PIPELINE

- 👉 How to launch a Spark environment (PySpark in Jupyter)
- 👉 How to create a Spark session
- 👉 How to run a Word Count program using RDDs

This is the "Hello World" of distributed data processing.

⚙️ 2. Step-by-step breakdown

🚀 Step 0: Launch environment

- Open Jupyter
 - Click PySpark3
 - This gives you a notebook connected to a Spark cluster
-

🔑 Step 1: Create Spark Session

Code shown:

```
from pyspark.sql import SparkSession
import getpass
```

```
username = getpass.getuser()
```

```
spark = SparkSession \
    .builder \
    .config("spark.ui.port", "0") \
    .config("spark.sql.warehouse.dir", f"/user/{username}/warehouse") \
    .enableHiveSupport() \
    .master("yarn") \
    .getOrCreate()
```

🧠 What this actually means (important)

◆ SparkSession

This is the entry point to Spark.

Think of it like:

“Your connection to the entire distributed cluster”

It internally contains:

- SparkContext (low-level engine)
 - SQL engine
 - Hive support
-

◆ getpass.getuser()

Gets your system username → used to create user-specific paths

◆ .builder

A builder pattern to configure Spark before starting it

◆ .config(...)

Used to set configurations:

- spark.ui.port = 0
→ Auto-assign UI port
 - spark.sql.warehouse.dir
→ Where Hive tables are stored
-

◆ .enableHiveSupport()

Allows Spark to use:

- Hive tables
 - Metastore
 - SQL queries
-

◆ .master("yarn")

Tells Spark:

"Run this on a Hadoop YARN cluster (not locally)"

◆ .getOrCreate()

- If Spark exists → reuse it
 - Else → create new session
-

🔍 After creation:

spark

You see:

- SparkSession
 - SparkContext
 - Version
 - Cluster info
-

⚡ 3. SparkContext (Important concept)

spark.sparkContext

What it is:

Low-level execution engine

Why needed:

RDD operations run through SparkContext

📁 4. Load Data into RDD

```
rdd1 = spark.sparkContext.textFile("<file-path>")
```

🧠 What happens internally:

- File is NOT loaded immediately !
- Spark creates a logical plan

This is called:

👉 Lazy Evaluation

Output:

RDD of lines:

```
["hello world", "hi python"]
```

5. Transformations (Core Logic)

Transformations are lazy operations

They build a DAG (execution graph)

◆ Step 5.1: flatMap

```
rdd2 = rdd1.flatMap(lambda x: x.split(" "))
```

What it does:

- Splits each line into words
 - Flattens the result
-

Example:

Input:

```
["hello world", "hi python"]
```

Output:

```
["hello", "world", "hi", "python"]
```

Why "flat" map?

- `map()` → returns nested list
 - `flatMap()` → flattens it
-

◆ Step 5.2: map

```
rdd3 = rdd2.map(lambda word: (word, 1))
```

What it does:

Convert each word into a key-value pair

Output:

```
[("hello",1), ("world",1), ("hi",1), ("python",1)]
```

◆ Step 5.3: reduceByKey

```
rdd4 = rdd3.reduceByKey(lambda x, y: x + y)
```

What it does:

Groups by key and sums values

Output:

```
[("hello",1), ("world",1), ("hi",1), ("python",1)]
```

If duplicates:

```
[("hello",2), ("world",1)]
```

🧠 Internals (VERY IMPORTANT)

reduceByKey:

- Performs local aggregation first (map-side combine)
- Then shuffle
- Then final aggregation

👉 This makes it efficient for big data

⚡ 6. Actions (Trigger execution)

◆ Option 1:

```
rdd4.collect()
```

⚠ Problem:

- Brings all data to driver
 - Can cause OutOfMemory error
-

◆ Option 2 (Recommended):

```
rdd4.saveAsTextFile("<output-path>")
```

Why:

- Saves distributed output to HDFS
 - Scalable
-

7. DAG (Execution Plan)

All transformations:

textFile → flatMap → map → reduceByKey

Spark builds a DAG (Directed Acyclic Graph)

Execution happens ONLY when:

👉 collect() or saveAsTextFile() is called

8. Small Python Example (shown in image)

```
data = ["hello world", "hi python"]
```

```
result = [word for line in data for word in line.split()]
```

```
print(result)
```

Output:

```
['hello', 'world', 'hi', 'python']
```

This mimics flatMap

9. Senior Data Engineer Explanation (Real Insight)

Let's go beyond basics.

Key Concepts You Must Understand

1. Lazy Evaluation

Nothing runs until an action is triggered

2. Transformations vs Actions

Type	Examples	Behavior
Transformation	map, flatMap	Lazy
Action	collect, save	Executes DAG

3. RDD vs DataFrame

This example uses RDD (low-level)

In real-world:

👉 Prefer DataFrames (optimized via Catalyst engine)

4. Shuffle (Critical concept)

Occurs in:

reduceByKey

- Data moves across nodes
 - Expensive operation
 - Needs optimization
-

5. Distributed Thinking

Each step runs:

- On multiple machines
 - In parallel
-

🧠 Mental Model

Think like this:

File → Split across cluster

- Each node processes chunk
 - Partial results combined
 - Final output
-



Super Simple Summary

This is a Spark Word Count program using PySpark.

◆ What happens overall:

1. Start Spark
 - Create a `SparkSession` → connects your code to a cluster
 2. Load data
 - Read a text file into an RDD (distributed dataset)
 3. Process data
 - `flatMap()` → split lines into words
 - `map()` → convert each word into (word, 1)
 - `reduceByKey()` → count occurrences of each word
 4. Run the job
 - `collect()` → bring results to driver (small data only)
 - OR
 - `saveAsTextFile()` → store results in distributed storage
-



Core ideas:

- RDD = distributed data
 - Transformations (map, flatMap) = lazy (no execution yet)
 - Actions (collect, save) = trigger execution
 - DAG = execution plan
 - Shuffle = data movement (expensive step)
-



Interview-Level Questions (with depth)

◆ Basic (must know)

1. What is a `SparkSession`?
 - Entry point to Spark; replaces `SparkContext` in newer APIs

2. Difference between `map()` and `flatMap()`?
 - `map` = 1-to-1
 - `flatMap` = 1-to-many + flatten
 3. What is an RDD?
 - Immutable distributed collection of data
 4. What is lazy evaluation in Spark?
 - Transformations are not executed until an action is called
 5. What is the difference between transformation and action?
 - Transformation builds DAG
 - Action executes DAG
-

◆ Intermediate (very important)

6. What happens internally when you call `reduceByKey()`?
 - Local aggregation → shuffle → global aggregation
 7. Why is `reduceByKey` better than `groupByKey`?
 - `reduceByKey` minimizes data transfer (map-side combine)
 8. What is a DAG in Spark?
 - Directed Acyclic Graph representing execution steps
 9. What is a shuffle and why is it expensive?
 - Data movement across nodes → network + disk cost
 10. Why can `collect()` cause issues?
 - Loads all data into driver → memory overflow
-

◆ Advanced (interview differentiator)

11. How does Spark achieve parallelism?
 - Data partitioning + distributed execution across executors
12. What is the role of `SparkContext`?
 - Low-level engine that communicates with cluster manager
13. What is the difference between RDD and DataFrame?
 - RDD = low-level, no optimization
 - DataFrame = optimized (Catalyst engine)
14. What is lineage in Spark?
 - History of transformations used to recompute data if lost
15. How would you optimize this word count program?
 -

- Use DataFrames instead of RDD
 - Avoid shuffle-heavy operations
 - Use proper partitioning
 - Cache intermediate data if reused
-

WORD COUNT IN PYSPARK (RDD LEVEL)

Sample file content

hello world

hello spark

Step-by-step execution

1 Read file → RDD

```
rdd = sc.textFile("file.txt")
```

 RDD looks like:

```
["hello world", "hello spark"]
```

2 Split lines into words → flatMap

```
words = rdd.flatMap(lambda line: line.split())
```

 Output:

```
["hello", "world", "hello", "spark"]
```

 Why flatMap?

- Each line → multiple words
 - Flatten into single list
-

3 Convert to (word, 1) → map

```
pairs = words.map(lambda word: (word, 1))
```

👉 Output:

```
[("hello",1), ("world",1), ("hello",1), ("spark",1)]
```

4 Aggregate counts → reduceByKey

```
counts = pairs.reduceByKey(lambda a, b: a + b)
```

👉 Output:

```
[("hello",2), ("world",1), ("spark",1)]
```

5 Show result

```
counts.collect()
```

⚡ Full Code

```
rdd = sc.textFile("file.txt")
```

```
result = (rdd
```

```
    .flatMap(lambda line: line.split())
```

```
    .map(lambda word: (word, 1))
```

```
    .reduceByKey(lambda a, b: a + b))
```

```
print(result.collect())
```

🧠 What's REALLY happening (important 🔥)

Execution Flow:

File → RDD → flatMap → map → reduceByKey → result

🔪 Internal Insight (INTERVIEW GOLD)

◆ flatMap

Splits data → increases records

◆ map

Transforms → creates key-value pairs

◆ reduceByKey

- Groups same keys
- Performs **local aggregation first (map-side combine)**
- Then shuffles less data → faster

💬 One-line summary

Read → split → map to (word,1) → sum by key

🚨 Common Mistake

Using groupByKey instead of reduceByKey

✗ Bad:

- Sends all data over network

✓ Good:

- reduceByKey → combines early → efficient

Simulate a real Spark cluster execution

🧠 Problem: Word Count (RDD)

```
rdd = sc.textFile("file.txt")
```

```
result = (rdd
```

```
    .flatMap(lambda line: line.split())
```

```
    .map(lambda word: (word, 1))
```

```
    .reduceByKey(lambda a, b: a + b))
```

💻 Cluster Setup (assume)

- 1 Driver
- 2 Executors
- File split into **2 partitions**

Partition 1 → Executor 1

Partition 2 → Executor 2

Input File

hello world

hello spark

STEP-BY-STEP EXECUTION

◆ **STEP 1: Driver creates DAG**

Driver analyzes transformations:

textFile → flatMap → map → reduceByKey

👉 Builds **DAG (Directed Acyclic Graph)**

◆ **STEP 2: DAG → Stages**

Stage 1 (No Shuffle):

textFile → flatMap → map

Stage 2 (Shuffle Stage):

reduceByKey

👉 Why split?

- reduceByKey requires **data movement (shuffle)**

◆ **STEP 3: Task Distribution**

Each partition = **1 task**

Executor Partition Task

Exec 1 Part 1 Task 1

Executor Partition Task

Exec 2 Part 2 Task 2

STAGE 1 EXECUTION (Parallel)

Executor 1 processes:

"hello world"

flatMap:

["hello", "world"]

map:

("hello",1), ("world",1)

Executor 2 processes:

"hello spark"

flatMap:

["hello", "spark"]

map:

("hello",1), ("spark",1)

KEY INTERVIEW POINT

Before shuffle → Local Combine (IMPORTANT )

Each executor does **mini reduce (combiner)**:

Exec 1:

("hello",1), ("world",1)

→ same (no duplicates locally)

Exec 2:

("hello",1), ("spark",1)

→ same

 If duplicates existed, they'd be combined here → reduces network traffic

STAGE 2: SHUFFLE (Network happens here)

Data is redistributed by key

All "hello" → one partition

All "world" → one partition

All "spark" → one partition

After shuffle:

Executor 1 gets:

("hello",1), ("hello",1)

Executor 2 gets:

("world",1), ("spark",1)

FINAL REDUCE

Executor 1:

("hello",1+1) → ("hello",2)

Executor 2:

("world",1)

("spark",1)

FINAL OUTPUT

("hello",2), ("world",1), ("spark",1)

FULL FLOW (Whiteboard Style)

Driver:

Build DAG → Split into stages → Assign tasks

Executors:

Stage 1:

Read → flatMap → map

Local Combine:

Reduce within partition

Shuffle:

Exchange data across executors

Stage 2:

Final reduceByKey

Interview Power Points (Say these!)

-  “reduceByKey performs **map-side aggregation** before shuffle”
 -  “Shuffle is the **most expensive operation**”
 -  “Stages are split at shuffle boundaries”
 -  “Each partition = one task”
 -  “DAG → Stages → Tasks → Execution”
-

One-line killer answer

Spark splits the job into stages, processes partitions in parallel, locally aggregates data, then shuffles and reduces to produce final results.